

TEORÍA PROGRAMACIÓN

datos

- **Estándares:** Los tipos de datos definidos por el lenguaje (primitivos o estándar) son provistos por el lenguaje.
- **Definidos por el usuario:** permiten definir nuevos tipos de datos a partir de los tipos simples

Los tipos de datos

- **simples** → si en un momento dado de la ejecución pueden contener sólo un valor de los posibles para el tipo.
- **ordinales** → si para cada valor de los posibles para el tipo, se puede decir cuál es su inmediato anterior y posterior.

Tipos de Datos - Numérico (Son tipos de datos simples)

- **Enteros** → Es simple y ordinal.
 - Los datos de este tipo no poseen decimales.
- **Reales** → Es simple
 - Permite representar números con decimales.
 - Los elementos de este tipo de datos se representan con cierta cantidad de bits para la parte entera y otra cierta cantidad para la parte decimal.

Constantes

- Se indica un valor en su declaración
- Su valor no cambia durante la ejecución del programa
- Se declara con la palabra clave **const** **const nombre = valor;**
- donde nombre es el identificador que representa el nombre de la constante. El tipo de dato de la constante queda definido implícitamente por el tipo de dato de valor. Se indica un valor en el programa

Variables

- Su valor cambia durante la ejecución del programa
- Se declara con la palabra clave **var** **var nombre : tipodedato;**
- donde nombre es el identificador que representa el nombre de la variable.

ESTRUCTURAS DE CONTROL

- **DECISIÓN** → **if**
- **SELECCIÓN** → **case**
 - Permite realizar una o más acciones dependiendo de cuál de todas las condiciones evaluadas es verdadera.
 - La variable de decisión debe ser de **tipo ordinal** Puede haber más de un valor en cada una de las entradas.
 - Un valor **debe aparecer en una sola de las entradas**.
 - Deben **incluirse todas las posibilidades**.
- **ITERACIÓN** → ejecutar un bloque de instrucciones desconociendo el número exacto de veces a realizar. **estructuras de control iterativas condicionales**.
 - **Precondicionales** → **while**
 - Evalúan la condición y luego ejecutan las acciones.
 - Realiza las acciones mientras la condición es verdadera.
 - Se ejecuta 0,1 o más veces.
 - **Postcondicionales**. → **repeat until**

Selección

- ☒ Simple **if**
- ☐ Múltiple **case**

Iterativas

- ☒ Precondicional **while**
- ☐ Postcondicional **until**
- ☐ Repetitiva **for**

- Ejecuta el bloque de acciones y luego evalúa la condición.
 - Realiza las acciones mientras la condición es falsa (o hasta que la condición se hace verdadera).
 - Se ejecuta 1 o más veces.
 - **REPETICIÓN → for**
 - repetir un número fijo y conocido de veces una o más acciones.
- Consideraciones acerca de la **variable índice** en Pascal:
- La variable de control debe ser de tipo ordinal (entero, boolean, char).
 - No debe modificarse dentro del lazo □ Error
 - Los incrementos o decrementos y testeos son implícitos
 - Al terminar el ciclo, la variable índice no tiene un valor definido

MODULARIZACIÓN

Significa dividir un problema en partes funcionalmente independientes, que encapsulen operaciones y datos.

Se busca obtener:

- **Alta Cohesión:** identificación de un módulo con una F concreta.
- **Bajo Acoplamiento:** medida de la interacción de los módulos que constituyen un programa.

Módulos

Conjunto de instrucciones que cumplen una tarea específica bien definida, se comunican entre sí adecuadamente y cooperan para conseguir un objetivo común.

- Cada módulo encapsula, acciones tareas o funciones
- Hay que representar los objetos relevantes del problema a resolver

Ventajas

Mayor productividad → Al dividir un sistema en módulos funcionalmente independientes, se puede trabajar simultáneamente en varios módulos, incrementando la productividad

Reusabilidad → Posibilidad de utilizar repetidamente el producto de software desarrollado.

Facilidad de Mantenimiento Correctivo → La división en módulos permite aislar los errores. Se pueden corregir los errores en menor tiempo y disminuyen los costos de mantenimiento de los sistemas.

Facilidad para el crecimiento del sistema → La modularización permite disminuir los riesgos y costos de incorporar nuevas prestaciones a un sistema en funcionamiento.

Mayor Legibilidad → Mayor claridad para leer y comprender el código fuente.

Alcance de variables

Es el contexto donde una variable puede ser referenciada o nombrada y ésta es reconocida.

- **Variable global:** su declaración se hace en la sección de declaración del programa principal, fuera de todos los módulos del programa y son accesibles desde cualquier parte del código.
NO ACONSEJABLE. PUEDE TRAER EFECTOS COLATERALES. PERJUDICA LA LEGIBILIDAD
- **Variable local a un módulo:** su declaración se hace en un módulo particular y sólo podrá ser usada por ese módulo. Si este módulo contiene a su vez otros módulos, entonces esa variable puede ser también usada por todos los módulos interiores.

- **Variable local al programa:** su declaración se hace antes de la sección de instrucciones ejecutables del programa y después de la declaración de los módulos del programa. Su uso se limita a la sección de instrucciones ejecutables.

TIEMPO DE VIDA DE UNA VAR: DESDE EL MOMENTO EN QUE SE RESERVA SU MEMORIA HASTA QUE SE LIBERA

COMUNICACIÓN ENTRE MÓDULOS

La comunicación externa de un módulo con el resto del sistema es aconsejable que se produzca a través de datos de entrada y datos de salida a través de parámetros.

Parámetros por valor

- el módulo recibe (sobre una var local) un valor de otro módulo (o del p. principal).
- Con él, el módulo puede realizar operaciones y/o cálculos, pero no producirá ningún cambio ni tampoco tendrá incidencia fuera del módulo.

Parámetros por referencia

- El módulo recibe la dirección de memoria de una var conocida en el punto de invocación.
- Dentro del módulo se puede operar con el valor original contenido en esa dirección de memoria, y las modificaciones que se produzcan se reflejan en los demás módulos que conocen la variable. En el encabezado del módulo tienen la palabra clave VAR.

Conceptos importantes

- El número y tipo de los parámetros o argumentos utilizados en la invocación a una Función o un Procedimiento deben coincidir con el N y tipo de parámetros del encabezamiento del módulo.
- Un parámetro por valor debiera ser tratado como una var de la cuál el Procedimiento o Función hace una copia y la utiliza localmente.
- Los parámetros por referencia operan directamente sobre la dirección de la var original. = No requiere memoria local.

Conclusiones

- Al modularizar es importante obtener independencia funcional de cada módulo. Disminuye el acoplamiento con el resto del sistema y reduce el impacto de las fallas y modificaciones.
- La comunicación entre módulos es por el intercambio de datos por parámetros.
- En todos los casos deben evitarse las variables globales.

Procedimientos y Funciones

Procedure puede retornar 0,1 o más valores. Puede ocurrir que un módulo Procedure contenga además otros módulos. Esto es un procedimiento con módulos anidados.

- Puede devolver resultados a través de parámetros por referencia.

Function retorna un único valor de tipo simple. Luego ejecutado el módulo, el flujo de control retorna a la misma instrucción donde fue invocado el módulo.

aspectos que los diferencian

- Encabezamiento del módulo
- Invocación
- Lugar donde retorna el flujo de control una vez ejecutado el módulo

Tipos de datos estándares

definidas y acotadas por el lenguaje.

- El conjunto de valores de ese tipo
- Las operaciones que se pueden efectuar
- Su representación

Tipos de datos definidos por el usuario

Es aquel que no existe en la definición del lenguaje, y el programador es el encargado de su especificación. **Declarados en el type** → `nuevoTipo = ... ;`

Ventajas

- riqueza expresiva del lenguaje, con mejores posibilidades de abstracción de datos.
- **Seguridad:** Evita errores al limitar las operac permitidas para cada tipo de dato.
- Límites preestablecidos sobre los valores posibles que pueden tomar las variables que corresponden al tipo de dato.
- **Flexibilidad:** en el caso de ser necesario modificar la forma en que se representa el dato, sólo se debe modificar una declaración en lugar de un conjunto de declaraciones de variables.
- **Documentación:** se pueden usar como identificador de los tipos, nombres autoexplicativos, facilitando el entendimiento y lectura del programa.

Subrangos → `años= 1960..1990;` `letrasMay= 'A'..'Z';`

- Consiste en una sucesión de valores de un tipo ordinal tomado como base.

Conjuntos → `identificador = set of ...;` `letras = set of char;` → `letras:= ['a'..'f'];`

- **Colección de datos simples** (datos guardados en el conjunto deben ser de tipo ordinal), sin repetición y limitada según cada lenguaje o sist. operativo.
- Es un tipo de datos compuesto.
- No es un tipo de datos ordinal.
- Se pueden tener conjuntos de valores enteros, boolean, y char.
- En la implementación de Pascal el conjunto no puede tener más de 255 elementos
- No permite operaciones de lectura - escritura.
- Permite las operaciones de asignación:
 - **unión** → `carac2:= ['a'] + carac1;`
En este conjunto resultado aparecen los elementos de los dos conjuntos y aquellos elementos repetidos aparecen una vez.
 - **intersección** → `carac2:= ['a'] * carac1;`
En el conjunto resultado aparecen solamente los elementos comunes a los dos conjuntos.
 - **diferencia** → `carac2:= ['a','E'] - carac1;`
Este conjunto resultado contiene los elementos que están en el primer conjunto y no están en el segundo.
 - **pertenencia** → `if ('a' IN carac1) then ...`
Esta operación devuelve verdadero si el elemento está en el conjunto y falso en caso contrario.
 - **comparación** → `if (carac1 <= carac2)`
Usamos los operadores relacionales para determinar si un conjunto está incluido en otro (<=), si son distintos (<>) o iguales (=).

String → `identificador = string[k];`

- Es una sucesión de caracteres, que se almacenan en un área de la memoria.
- Existe en la mayoría de los lenguajes como un tipo predefinido.
- No es un dato simple.

ESTRUCTURAS DE DATOS

conjunto de variables (podrían ser de distinto tipo) relacionadas entre sí y que se puede operar como un todo, bajo un nombre único.

Clasificar

De acuerdo al **tipo de datos** que la componen:

- **homogénea** si los datos que la componen son todos del mismo tipo
- **heterogénea** si los datos que la componen son de distinto tipo.

De acuerdo a la **ocupación de memoria** las estructuras pueden ser:

- **estática** si la cantidad de elementos que contiene es fija. La cantidad de memoria que ocupa no varía durante la ejecución del programa.
- **dinámica** si la cantidad de elementos que contiene es variable, la cantidad de memoria ocupada puede cambiar durante la ejecución de un programa.

De acuerdo al acceso a sus elementos:

- de **acceso secuencial**, si para acceder a un elemento particular se debe respetar un orden predeterminado, **por ej, pasando por todos los elementos que le preceden.**
- de **acceso directo**, si se puede acceder a un elemento particular, directamente, sin necesidad de pasar por los anteriores a él, **por ej, indicando una posición.**

De acuerdo a la relación entre sus elementos:

- **lineal** cuando está formada por ninguno, uno o varios elementos que guardan una relación de adyacencia ordenada donde a cada elemento le sigue uno y le precede uno, solamente.
- **No lineal** si para un elemento dado pueden existir 0, 1 o más elementos que le suceden y 0, 1 o más elementos que le preceden.

Estructura de datos Registro

Características :

- Los valores pueden ser de diferentes tipos, esto convierte a un registro en una estructura de datos heterogénea.
- El almacenamiento ocupado por un registro es fijo, por esto, un registro es una estructura estática.
- El acceso a sus componentes (campos) es directo. Debe referenciarse a través de su nombre.

Estructura de datos ARREGLO

Colección de elementos que se guardan consecutivamente en la memoria y se pueden referenciar a través de índices

Características:

- Todos los elementos son del mismo tipo de datos, por eso es una estructura de datos **homogénea**.
- Los elementos o componentes pueden recuperarse en cualquier orden, indicando simplemente su pos, → es una estructura de datos de **acceso directo**. Como el acceso se hace a través del índice se la denomina también **indexada**.
- La memoria ocupada durante la ejecución del programa es fija, → es una estructura de datos **estática**.
- Como cada elemento tiene un elemento que le precede y uno que le sigue, se denomina **lineal**.

Vectores Vector = Array [índice] of tipo_elementos;

Es posible indexar los elementos por un índice que corresponde a cualquier tipo ordinal:

- **Entero • Carácter • Subrango**

Los elem de un arreglo pueden pertenecer a cualquier tipo de datos de asignación estática:

- Entero, Real, Lógico, Carácter • String • Registros • Otro arreglo

Operaciones:

- Asignación de contenido a un elemento
- Lectura / Escritura
- **Recorridos** → recorrer el vector para realizar algún proceso sobre sus elementos.
 - **Total**, implica analizar todos los elementos del vector, se recorre completamente la estructura
 - **Parcial**, analiza los elementos del vector, hasta encontrar aquel que cumple con lo pedido.
- **Cargar** datos en un vector
 - Carga Completa → guardar un elemento en cada posición del vector
 - Considerar: dimF, dimL, espacio suficiente, condición de corte
- **Agregar al final**
 - Incorporar el elemento a continuación del último ingresado, en la posición siguiente a la indicada en la dimL.
 - Considerar: dimF, dimL, espacio suficiente, nue elem pos dimL+1, dimL+1
- **Insertar** elementos
 - En una **pos determinada**
 - verificar que la posición sea válida.
 - En un **orden determinado**.
 - Considerar: dimF, dimL, espacio suficiente, dimL:= dimL+1
- **Borrar** elementos
 - De una **pos determinada**
 - verificar que la pos sea válida.
 - un **elemento** determinado
 - Requiere primero buscar el elemento y luego borrarlo
 - Considerar: dimF, dimL, espacio suficiente, dimL:= dimL+1
- **Buscar** un elemento
 - **Lineal o secuencial (elementos sin orden)** → La búsqueda comienza desde el principio y se avanza por la estructura de manera secuencial, hasta encontrar el elemento.
 - Requiere excesivo consumo de tiempo en la localización del elemento
 - Es ineficiente.
 - **Secuencial Optimizado (elementos con orden)** → Comienza desde el principio y se avanza por la estructura de manera secuencial y de a uno hasta encontrar el número buscado o uno mayor
 - **Binaria o Dicotómica (elementos con orden)** → Se compara el valor buscado (x) con el ubicado en el medio del vector (a): Si el elemento ubicado al medio del vector es igual a x, entonces la búsqueda termina. Si no es el valor buscado, debería quedarse con la mitad del vector que conviene, para seguir la búsqueda. Este paso se repite hasta que se acaba el vector o encuentro el valor.

Dimensión Física del vector → Se especifica en el momento de la declaración y determina su ocupación de memoria. La cantidad de memoria no variará durante la ejecución.

Dimensión Lógica → Se determina cuando se cargan elementos al vector. Indica la cantidad de posiciones de memoria ocupadas con contenidos cargados desde la pos inicial en forma consecutiva.

Matriz

estructura de datos compuesta que permite acceder a cada componente utilizando dos índices (fila y columna)

Características: Homogénea Estática Acceso directo

Matriz = Array [fila, columna] of tipo_elementos;

Es posible indexar los elementos por un índice que corresponde a cualquier tipo ordinal: -

- Entero - Carácter - Subrango

Los elementos de un arreglo pueden pertenecer a cualquier tipo de datos:

- Enteros - Reales - Caracteres - Registros - String - Lógicos - Otros

tener en cuenta:

- DimF de filas - DimL de filas / DimF de columnas - DimL de columnas

PUNTEROS

- Una variable de tipo puntero ocupa una cant de memoria estática fija (4 bytes), independiente del tipo de dato al que apunta.
- Un dato referenciado o apuntado no tienen memoria asignada
- Para emplear variables dinámicas es necesario usar punteros que permiten referenciar nuevas pos de memoria que no han sido utilizadas a priori y que se van a crear y destruir en tiempo de ejecución.
- Las **vars dinámicas** son aquellas que se crean cuando se necesitan y se destruyen cuando ya han cumplido con su cometido.
- En Pascal la creación y destrucción de variables dinámicas se realiza mediante los procedimientos **New (puntero)**, **Dispose (puntero)**
- Un puntero es una variable que almacena direcciones de memoria.
- Contiene la dirección donde reside un valor (de cualquier tipo: char, boolean, integer, real, string, registro, arreglo u otro puntero).
- Es un tipo de dato simple.
- Apunta solo a direcciones en memoria dinámica (heap).
- Puede apuntar a un solo tipo de dato.

Operaciones

- Asignación de un valor a una variable puntero
- Asignación de valor al objeto "referenciado" o "apuntado" por el puntero
- Acceso a la información del objeto "referenciado" o "apuntado" por el puntero
- Operaciones de Entrada / Salida
- Operaciones de comparación
- Eliminación de un objeto apuntado que no se necesita

LISTAS

- Colección de elementos homogéneos, con una relación lineal, **cada elemento tiene un único predecesor (excepto el primero), y un único sucesor (excepto el último)**
- Los elementos que la componen no ocupan posiciones secuenciales o contiguas de memoria. **Pueden aparecer dispersos en la memoria dinámica, pero mantienen un orden lógico interno.**
- Se accede a sus elementos secuencialmente

Características:

- Están compuestas por nodos.
- Los nodos se conectan por medio de enlaces o punteros.
- Cuando se necesitan agregar nodos a la estructura, se solicita espacio adicional.
- Cuando existen nodos que ya no se necesitan, pueden ser borrados, liberando memoria.
- Siempre se debe conocer la dirección del primer nodo de la lista (puntero inicial) para acceder a la información de la misma.
- El último nodo de la lista se caracteriza por tener su enlace en Nil.

Operaciones:

- Crear una lista vacía
- Agregar un elemento al principio de una lista
- Recorrer una lista
- Buscar un elemento en la lista
- Agregar un elemento al final de una lista
- Eliminar un elemento de la lista
- Insertar un nuevo elemento en una lista ordenada

Listas con Doble Enlace

Podemos recorrerla en 2 sentidos (órdenes) diferentes.

ventaja: Pueden accederse siguiendo cualquiera de los 2 órdenes, sin utilizar espacio extra..

inconveniente: Ocupan más memoria por nodo que una lista simple

Listas Doblemente Enlazadas

Guardan anterior y siguiente. Los enlaces permitirán recorrer la lista en dos sentidos de manera tal que un sentido da la inversa del otro.

EFICIENCIA

Cálculo de la memoria utilizada

Se puede calcular antes de la ejecución únicamente la cantidad de memoria estática que utiliza el programa. Se deben analizar las variables declaradas y el tipo correspondiente.

Cálculo del tiempo

El tiempo de ejecución de un programa puede calcularse de dos maneras:

- **EMPÍRICO:** es necesario realizar el programa y medir los recursos consumidos.
 - Tiene limitaciones: puede dar una info pobre de los recursos consumidos si el programa es aplicado a pruebas de escritorio.
- **TEÓRICO:**
 - Reglas Generales
 - Regla 1: For
 - Regla 2: For anidados
 - Regla 3: sentencias consecutivas
 - Regla 4: If / else
 - Se tendrá en cuenta como una medida de eficiencia en cuanto al tiempo de ejecución el N de **operaciones elementales** que emplea el algoritmo.
 - Una operación elemental utiliza **un tiempo** constante para su ejecución, independientemente del tipo de dato con el que trabaje.
 - Se considera que cada operación elemental se ejecuta en una unidad de tiempo.
 - Una operación elemental es

- una asignación, (1UT)
- una comparación (1UT)
- una operación aritmética simple (1UT)

El tiempo total para este algoritmo sería: $1UT + 1UT + 1UT = 3UT$.

Algoritmos eficientes:

Algoritmo A.

```
m:=a
if b<m then m := b;
if c<m then m := c;
```

Algoritmo B.

```
if(a<=b) and (a<=c)
then m:=a
else if b<=c then m := b
else m := c;
```

El algoritmo A es más eficiente ya que realiza dos comparaciones y al menos una asignación, el algoritmo B, puede llegar a hacer 3 comparaciones y una asignación

Debemos evitar realizar cálculos innecesarios:

```
y:=0;
for n:=1 to 2000 do
y:= y+1/(x*x*x-n)
```

Este algoritmo realizaría el cálculo de $x*x*x-n$ 2000 veces, cosas que podemos evitar haciendo el siguiente algoritmo:

```
t:= x*x*x;
y:=0
for n=1 to 2000 do
y:=y+1/(t-n);
```

Una instrucción elemental utiliza un tiempo constante para su ejecución. Cada una de ellas se ejecuta en una unidad de tiempo (1T).

Ejemplo 1:

Begin

```
aux:= 58; //Asignación = 1T.
aux:= aux * 5; //Multiplicación +
asignación = 2T.
temp:= aux; //Asignación = 1T.
read (x); //NO SE CONSIDERA.
```

End.

Ejemplo 2:

if (aux > 45) //Evaluar condición = 1T.

Ejemplo 3:

for i:= 1 to 10 do

- Tiempo del for = $(3(N) + 2) + N(\text{cuerpo})$
- $N = 10$.
- (cuerpo) = (suma + asignación = 2UT) + multiplicación + asignación = 2UT)

x:= x + 1;

temp:= aux * 2;

Ejemplo 4:

while (aux > 0) do // Tiempo del while $N+1$ (eval. cond.)+N(cuerpo)

CADP – EFICIENCIA DE PROGRAMAS T. de EJECUCION

ANÁLISIS TEÓRICO

Program uno;

var
aux,temp,x: integer;

Begin
(1) aux:= 58;
(2) aux:= aux * 5;
(3) temp:= aux;
(4) read (x);
End.

El tiempo de ejecución de un algoritmo que **NO** tiene estructuras de control está dado por:

$T(\text{alg}) = T(1) + T(2) + T(3) + T(4)$

$T(1) =$ asignación = 1UT

$T(2) =$ multiplicación + asignación = 2UT

$T(3) =$ asignación = 1UT

$T(4) =$ no se considera

T (alg) = 4UT

Qué ocurre cuando hay estructuras de control?

CADP – EFICIENCIA DE PROGRAMAS T. de EJECUCION

El tiempo de ejecución del **IF** → Tiempo de evaluar la condición + tiempo del cuerpo. Si hay else, Tiempo de evaluar la condición + max(then,else).

El tiempo de ejecución del **FOR** → $(3N + 2) + N(\text{cuerpo del for})$. N representa la cantidad de veces que se ejecuta el for.

El tiempo de ejecución del **WHILE** → $C(N+1) + N(\text{cuerpo del while})$. N representa la cantidad de veces que se ejecuta el while. (N>0) C cantidad de tiempo en evaluar la condición

El tiempo de ejecución del **REPEAT UNTIL** → $C(N) + N(\text{cuerpo del repeat})$. N representa la cantidad de veces que se ejecuta el repeat. (N>0) C cantidad de tiempo en evaluar la condición

ANALISIS TEORICO

$$T(\text{alg}) = T(1) + T(2) + T(3) + T(4)$$

Program uno;
var
aux,temp,x: integer;

Begin
(1) aux:= 58;
(2) aux:= aux * 5;
(3) if (aux > 45) and (aux < 300) then
begin
temp:= aux - 5;
x:= temp + aux + 2;
end;
(4) x:= x * 10;
end;

T(1)= asignación = 1UT

T(2)= multiplicación +
asignación = 2UT

T(3)= IF= evaluar la condición
+ cuerpo=
(1) + (1) + (conector) = 3 +
resta + asignación
suma + suma + asignación
= 3UT + 2UT + 3UT = 8UT

T(4)= multiplicación +
asignación = 2UT

$$T(\text{alg}) = 13 \text{ UT}$$

Clase 11-2

ANALISIS TEORICO

$$T(\text{alg}) = T(1) + T(2)$$

Program uno;
var
aux,temp,x: integer;

Begin
(1) read(aux);
(2) if (aux > 45) then
begin
temp:= aux - 5;
x:= temp;
end
else
aux:= aux + 1 * (aux MOD 2);
end;

T(1)= el read no se tiene
en cuenta.

T(2)= IF= evaluar la condición
+ max(then,else)
cond= 1 +
then= 2 + 1 = 3
else= 4
= 1UT + max(3,4) = 5UT

$$T(\text{alg}) = 5 \text{ UT}$$

Clase 11-2

ANALISIS TEORICO

$$T(\text{alg}) = T(1) + T(2) + T(3)$$

Program uno;
var
i,temp,x: integer;

Begin
(1) aux:= 8;
(2) for i:= 1 to 5 do
begin
x:= aux;
aux:= aux + 5;
end
(3) aux:= aux + 1;
end;

T(1)= asignación 1 UT.

T(2)= for= $3N+2 + N(\text{cuerpo})$

N= 5
(3*5) + 2 + 5(cuerpo)
17 + 5(cuerpo)
cuerpo = 1 + 2 = 3UT
>> 17 + 5(3) =
= 17 + 15 = 32UT

T(3)= 1 + 1 = 2 UT.

$$T(\text{alg}) = 35 \text{ UT}$$

Clase 11-2

ANALISIS TEORICO

$$T(\text{alg}) = T(1) + T(2) + T(3)$$

Program uno;
var
i,temp,x: integer;

Begin
(1) aux:= 8;
(2) for i:= 4 to 9 do
begin
x:= aux;
aux:= aux + 5;
end
(3) aux:= aux + 1;
end;

T(1)= asignación 1 UT.

T(2)= for= $3N+2 + N(\text{cuerpo})$

N= 6 (LS-LI + 1)
(3*6) + 2 + 6(cuerpo)
20 + 6(cuerpo)
cuerpo = 1 + 2 = 3UT
>> 20 + 6(3) =
= 20 + 18 = 38UT

T(3)= 1 + 1 = 2 UT.

$$T(\text{alg}) = 41 \text{ UT}$$

Clase 11-2

ANALISIS TEORICO

$$T(\text{alg}) = T(1) + T(2) + T(3)$$

Program uno;
var
i,temp,x: integer;

Begin
(1) aux:= 0;
(2) while (aux < 5) do
begin
x:= aux;
aux:= aux + 1;
end

T(1)= asignación 1 UT.

T(2)= while= $C(N+1) + N(\text{cuerpo})$

C= 1
N= 5
1*(6) + 5(cuerpo)
cuerpo = 1 + 2 = 3UT
>> 6 + 5(3) =
= 6 + 15 = 21UT

T(3)= 1 + 1 = 2 UT.

$$T(\text{alg}) = 24 \text{ UT}$$

Clase 11-2

ANALISIS TEORICO

$$T(\text{alg}) = T(1) + T(2) + T(3)$$

Program uno;
var
i,temp,x: integer;

Begin
(1) read(aux);
(2) while (aux < 5) do
begin
x:= aux;
aux:= aux + 1;
end

T(1)= read no se cuenta 0 UT.

T(2)= while= $C(N+1) + N(\text{cuerpo})$

C= 1
N= ???
1*(N+1) + N(cuerpo)
cuerpo = 1 + 2 = 3UT
>> N+1 + N(3) =
= N + 1 + 3N = 4N + 1 UT

T(3)= 1 + 1 = 2 UT.

$$T(\text{alg}) = 0 + 4N + 1 + 2 = (4N + 3) \text{ UT}$$

Clase 11-2

ANALISIS TEORICO

$$T(\text{alg}) = T(1) + T(2) + T(3)$$

Program uno;
var
i,temp,x: integer;

Begin
(1) aux:=0;
(2) while (aux >= 0) and (aux<5) do
begin
x:= aux;
aux:= aux + 1;
end

T(1)= 1 UT.

T(2)= while= $C(N+1) + N(\text{cuerpo})$

C= (1) + (1) + conector = 3
N= 5
3*(5+1) + 5(cuerpo)
cuerpo = 1 + 2 = 3UT
>> 18 + 5(3) =
= N + 18 + 15 = 33 UT

T(3)= 1 + 1 = 2 UT.

$$T(\text{alg}) = 1 + 33 + 2 = 36 \text{ UT}$$

Clase 11-2

ANALISIS TEORICO

$$T(\text{alg}) = T(1) + T(2) + T(3)$$

Program uno;
var
i,temp,x: integer;

Begin
(1) aux:=0;
(2) repeat
begin
x:= aux;
aux:= aux + 1;
until (aux > 5)
(3) aux:= aux + 1;
end;

T(1)= 1 UT.

T(2)= repeat= $C(N) + N(\text{cuerpo})$

C= 1
N= 5
1*(5) + 5(cuerpo)
cuerpo = 1 + 2 = 3UT
>> 5 + 5(3) =
= N + 5 + 15 = 20 UT

T(3)= 1 + 1 = 2 UT.

$$T(\text{alg}) = 1 + 20 + 2 = 23 \text{ UT}$$

Clase 11-2

MEMORIA :

La memoria necesaria para la ejecución del programa se divide en:

- Memoria **Estática**: Incluye variables locales y globales del programa.
- Memoria **Dinámica**: Engloba la reserva y liberación de memoria en tiempo de ejecución.

Tabla de Ocupación:

- | | | |
|---------------------|---------------------------------|-------------------------------------|
| • char (1 byte) | • string (tamaño + 1 byte) | • arreglos (dimF* tipo de elemento) |
| • boolean (1 byte) | • subrango (depende del tipo) | • puntero (4 bytes) |
| • integer (4 bytes) | • registro (suma de sus campos) | |
| • real (8 bytes) | | |

EJEMPLOS

Cálculo de memoria (Estática y dinámica):

Program dos;

Type

```
puntero = ^real;
puntero2 = ^char;
persona = record
    nombre:string[20];
    dni:integer;
end;
```

Var

```
p: puntero; //LOS 4 BYTES DECLARADOS (YA QUE SON DE TIPO PUNTERO).
q: puntero2; //LOS 4 BYTES DECLARADOS (YA QUE SON DE TIPO PUNTERO).
per: persona; //LOS 21 + 4 BYTES DECLARADOS.
precio: real; //8 BYTES DECLARADOS.
```

Begin

```
new (p); //8 BYTES DE DINÁMICA, YA Q SE HACE EL NEW() DE UN TIPO REAL.
```

End

TOTAL MEMORIA ESTÁTICA: 37 BYTES.

TOTAL MEMORIA DINÁMICA: 8 BYTES.

CADP – ALOCACION DE MEMORIA



Tipo de variable	Bytes que ocupa
Char	1 byte
Boolean	1 byte
Integer	6 bytes
Real	8 bytes
String	Tamaño + 1
Subrango	Depende el tipo
Registro	La suma de sus campos
Vector	Dimensión física * tipo elemento
Puntero	4 bytes

```
Type
vector = array[1..5] of real;
Var
v:vector;
letra:char;
num:integer;
ok:boolean;
p:punteroAEntero; //ya veremos como
```



Al comenzar mi programa ocupa

v = 5*8 = 40 bytes

letra = 1 byte

num = 6 bytes

ok = 1 byte

p = 4 bytes

**52 bytes de memoria
estática**

Si durante la ejecución del programa p **reserva** memoria se ocuparán tantos bytes de memoria dinámica como sea el contenido de p (en este caso 6 bytes de memoria dinámica). Luego p podrá **liberar** esa memoria dinámica durante la ejecución del programa.

CADP – MEMORIA DE UN PROGRAMA



```
Program uno;
Const
  DF = 10;
Type
  puntero = ^real;
  puntero2 = ^char;
  persona = record
    nombre:string[20];
    dni:integer;
  end;
  puntPer = ^persona;

  Var
    p:puntero; q:puntero2;
    per: puntPer;
    perso:persona;

Begin
End.
```

Tabla de ocupación:

char,	(1 byte)
boolean,	(1 byte)
integer	(4 bytes)
real,	(8 bytes)
string,	(tamaño + 1 byte)
subrango,	(depende el tipo)
registro,	(suma de sus campos)
arreglos	(dimFísica*tipo elemento)
puntero	(4 bytes)

CALCULO DE MEMORIA ESTATICA

DF = 4 bytes	
p = 4 bytes	
q = 4 bytes	
per = 4 bytes	
perso = 21 + 4 = 25 bytes	41 bytes

CALCULO DE MEMORIA DINAMICA

Como no hay operaciones de new() ni dispose(), no se modifica la memoria dinámica	0 bytes
---	---------

```
per: puntPer;
perso:persona;
Begin
  new(p);
End.

perso:persona;
Begin
  new(p);
  new(per);
End.

perso:persona;
Begin
  new(p);
  new(per);
  read (per^.nombre);
End.
```

CALCULO DE MEMORIA DINAMICA

Al hacer un new se reserva memoria para el contenido de p	8 bytes
--	---------

CALCULO DE MEMORIA DINAMICA

Al hacer un new se reserva memoria para el contenido de p y per (8 + 25)	33 bytes
--	----------

CALCULO DE MEMORIA DINAMICA

Al hacer un new se reserva memoria para el contenido de p y per (8 + 25), el read NO altera la memoria	33 bytes
---	----------

Clase 8-3

```
Program uno;
Const
  DF = 10;
Type
  puntero = ^real;
  puntero2 = ^char;
  persona = record
    nombre:string[20];
    dni:integer;
  end;
  puntPer = ^persona;

  Var
    p,p1:puntero; q:puntero2;
    per: puntPer;
    perso:persona;

Begin
  new(p);
  p1:= p;
End.
```

char, (1 byte)
boolean, (1 byte)
integer (4 bytes)
real, (8 bytes)
string, (tamaño + 1 byte)
subrango, (depende el tip
registro, (suma de sus ca
arreglos (dimFísica*tip
puntero (4 bytes)

CALCULO DE MEMORIA ESTATICA

DF = 4 bytes	
p,p1 = 8 bytes	
q = 4 bytes	
per = 4 bytes	
perso = 21 + 4 = 25 bytes	45 bytes

CALCULO DE MEMORIA DINAMICA

Al hacer un new se reserva memoria para el contenido de p, la asignación de direcciones NO altera la memoria	8 bytes
---	---------

```
Program uno;
Const
  DF = 10;
Type
  puntero = ^real;
  puntero2 = ^char;
  persona = record
    nombre:string[20];
    dni:integer;
  end;
  puntPer = ^persona;

  Var
    p:puntero; q:puntero2;
    per: puntPer;
    perso:persona;

Begin
  new(p);
  dispose(p);
End.

  Var
    p:puntero; q:puntero2;
    per: puntPer;
    perso:persona;

Begin
  new(p);
  p:= nil;
End.
```

char, (1 byte)
boolean, (1 byte)
integer (4 bytes)
real, (8 bytes)
string, (tamaño + 1 byte)
subrango, (depende el tip
registro, (suma de sus ca
arreglos (dimFísica*tip
puntero (4 bytes)

CALCULO DE MEMORIA ESTATICA

DF = 4 bytes	
p = 4 bytes	
q = 4 bytes	
per = 4 bytes	
perso = 21 + 4 = 25 bytes	41 bytes

CALCULO DE MEMORIA DINAMICA

Al hacer un new se reserva memoria para el contenido de p, luego al hacer dispose(p) se libera la memoria	0 bytes
--	---------

perso = 21 + 4 = 25 bytes

CALCULO DE MEMORIA DINAMICA

Al hacer un new se reserva memoria para el contenido de p, luego al hacer nil NO se libera la memoria	8 bytes
--	---------

```
Program uno;
Type
  puntero = ^real;
  puntero2 = ^char;
  persona = record
    nombre:string[20];
    dni:integer;
  end;
  puntPer = ^persona;

  Var
    p,p1:puntero;
    per: puntPer;

Begin
  new(per);
  new(p);
  p^:= 8;
  p1:= p;
  dispose(p1);
End.
```

char, (1 byte)
boolean, (1 byte)
integer (4 bytes)
real, (8 bytes)
string, (tamaño + 1
subrango, (depende e
registro, (suma de s
arreglos (dimFísica
puntero (4 bytes)

CALCULO DE MEMORIA ESTATICA

p = 4 bytes	
p1 = 4 bytes	
per = 4 bytes	12 bytes

CALCULO DE MEMORIA DINAMICA

new(per) = 25 bytes	
new(p) = 8 bytes	
dispose(p1) = -8 bytes	25 bytes